

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1-ANALYTICAL PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	25
CO2Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:		Reg.No.:	
Department:		Date:	

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.
2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.
3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.
4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.
5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Code of Conduct:

IR . M A N O J K U M A R N A I K (1 9 2 5 2 4 4 3 6) (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:**MARKS ALLOCATION**

	Problem Understanding(20%)	Method Selection(20%)	Solution Process(25%)	Accuracy of Calculation(20%)	Interpretation of Results(15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

ANSWERS

Question 1: Signed 8-bit Addition and Subtraction (+45 and -23)

1. Problem Understanding

The processor uses 8-bit 2's complement representation to perform arithmetic on signed integers.

Given:

- $A = +45$
- $B = -23$

Required:

- Convert to binary.
- Perform addition and subtraction.
- Show intermediate steps.
- Check overflow.
- Interpret the result.

2. Method Selection

Method used:

- Decimal $\rightarrow$ 8-bit Binary
- 2's Complement for negative number
- Binary Addition
- Binary Subtraction using 2's Complement
- Overflow Detection Rule

3. Solution Process

Step 1: Convert +45 into Binary

$45 = 00101101$

Step 2: Convert -23 into Binary

$23 = 00010111$

1's Complement = 11101000

2's Complement = 11101001

Therefore,

$-23 = 11101001$

A) Addition ($+45 + (-23)$)

00101101
 $+ 11101001$

 $1\ 00010110$

Ignore carry out.

Result:

00010110

Decimal = 22

Overflow Check

Signs:

$+45 \rightarrow$ Positive

$-23 \rightarrow$ Negative

Since operands have different signs,

Overflow = No

B) Subtraction ($+45 - (-23)$)

$= 45 + 23$

23 Binary

00010111

Now

00101101

$+$

00010111

01000100

Decimal

68

Overflow Check

Positive + Positive = Positive

No overflow.

4. Accuracy of Calculation

Operation	Binary Result	Decimal
-----------	---------------	---------

+45 + (-23)	00010110	22
-------------	----------	----

+45 - (-23)	01000100	68
-------------	----------	----

No overflow occurs in either operation.

5. Interpretation of Results

- The processor stores signed numbers using 2's complement.
- Overflow is detected when adding two numbers with the same sign produces a result with a different sign.
- Since the operands either have different signs or the result remains positive, no overflow occurs.
- Final results:
 - Addition = 22
 - Subtraction = 68
- This demonstrates correct arithmetic processing using 8-bit signed representation.

Question 2: Design a 4-bit Carry Look-Ahead (CLA) Adder

1. Problem Understanding

A 4-bit Carry Look-Ahead (CLA) Adder is designed to perform binary addition faster than a Ripple Carry Adder by calculating all carry bits simultaneously using Generate (G) and Propagate (P) signals.

Given:

- A = 1011
- B = 0110
- Initial Carry ($C_0 = 0$)

Required:

- Find Generate (G) and Propagate (P)
- Calculate all carry bits
- Determine the Sum
- Verify the result

2. Method Selection

The CLA adder uses the following equations:

Generate

$$G_i = A_i \times B_i$$

Propagate

$$P_i = A_i \oplus B_i$$

Carry Equations

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$C_4 = G_3 + P_3 C_3$$

Sum

$$S_i = P_i \oplus C_i$$

3. Solution Process

Step 1: Write Inputs

A = 1011

B = 0110

Bit A B

3 1 0

2 0 1

1 1 1

0 1 0

Step 2: Calculate Generate (G)

$$G = A \cdot B$$

Bit Calculation G

0 1×0 0

1 1×1 1

2 0×1 0

3 1×0 0

Therefore

G = 0010

Step 3: Calculate Propagate (P)

$$P = A \oplus B$$

Bit Calculation P

0 $1 \oplus 0$ 1

1 $1 \oplus 1$ 0

2 $0 \oplus 1$ 1

3 $1 \oplus 0$ 1

Therefore

P = 1101

Step 4: Calculate Carry

Initial Carry

C0 = 0

Carry 1

C1 = G0 + P0C0

=0+(1×0)

=0

Carry 2

C2 = G1 + P1G0 + P1P0C0

=1+(0×0)+(0×1×0)

=1

Carry 3

C3 = G2 + P2G1 + P2P1G0 + P2P1P0C0

=0+(1×1)+(1×0×0)+(1×0×1×0)

=1
 Carry 4
 $C4 = G3 + P3C3$

=0+(1×1)

=1
 Final Carry
 $C4 = 1$
 Step 5: Calculate Sum
 $S0 = P0 \oplus C0$

=1⊕0

=1
 $S1 = P1 \oplus C1$

=0⊕0

=0
 $S2 = P2 \oplus C2$

=1⊕1

=0
 $S3 = P3 \oplus C3$

=1⊕1

=0
 Final Sum
 10001
 Verification
 Decimal values:
 $1011_2 = 11$

$0110_2 = 6$
 Addition
 $11 + 6 = 17$
 Binary
 $17 = 10001_2$
 Verified
 4. Accuracy of Calculation

Parameter	Result
Generate (G)	0010
Propagate (P)	1101
Carry (C4)	1
Sum	10001
Decimal Result	17

5. Interpretation of Results

- The **Carry Look-Ahead Adder** computes carry bits in parallel, making it significantly faster than a Ripple Carry Adder.
- Generate and Propagate signals eliminate the need to wait for each carry to ripple through every stage.
- The final result is **10001₂**, which equals **17₁₀**, confirming the correctness of the design.
- This demonstrates the high-speed addition capability of the CLA architecture, making it suitable for modern processors and digital arithmetic circuits

Question 3: Design a 4-to-16 Decoder Using 2-to-4 Decoders

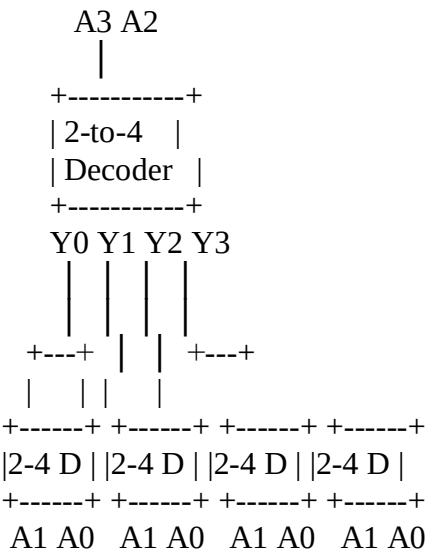
1. Problem Understanding

Design a **4-to-16 decoder** using **2-to-4 decoders**. A decoder converts a binary input into one active output.

2. Method

- Use **5 × 2-to-4 decoders**.
- One decoder is used for the two most significant bits (A3, A2).
- Four decoders are used for the two least significant bits (A1, A0).
- Outputs of the first decoder act as enable signals for the remaining four decoders.

3. Circuit Structure



4. Output Mapping

Inputs (A3 A2 A1 A0) Active Output

0000	Y0
0001	Y1
0010	Y2
0011	Y3
0100	Y4
0101	Y5
0110	Y6
0111	Y7
1000	Y8
1001	Y9

Parameter

Result

1010	Y10
1011	Y11
1100	Y12
1101	Y13
1110	Y14
1111	Y15

5. Result

- **Number of 2-to-4 decoders required: 5**
- **Inputs: 4**
- **Outputs: 16**
- Only **one output is HIGH** for each input combination.

Final Answer: A **4-to-16 decoder** can be implemented using **five 2-to-4 decoders**, where one decoder generates enable signals and the remaining four produce the sixteen output lines.

Question 4: Design a 4:1 Multiplexer Using 2:1 Multiplexers

1. Problem Understanding

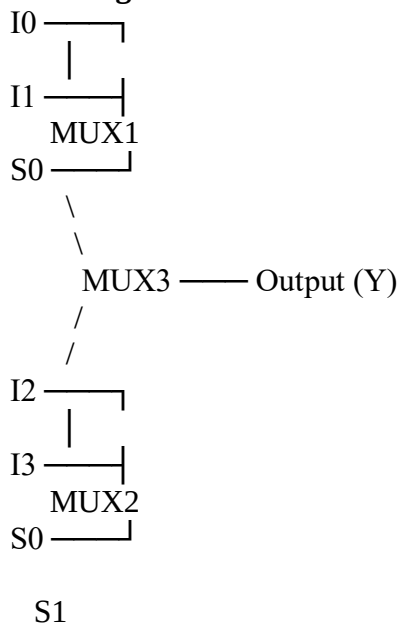
Design a **4:1 Multiplexer (MUX)** using only **2:1 Multiplexers**. A 4:1 MUX selects one of four inputs using two select lines.

2. Method

Use **3 × 2:1 Multiplexers**.

- **MUX1:** Selects between **I₀** and **I₁** using **S₀**.
- **MUX2:** Selects between **I₂** and **I₃** using **S₀**.
- **MUX3:** Selects between the outputs of MUX1 and MUX2 using **S₁**.

3. Circuit Diagram



4. Truth Table

S₁ S₀ Output (Y)

0	0	I ₀
0	1	I ₁

Parameter

Result

1 0 I_2

1 1 I_3

5. Accuracy of Design

- Number of **2:1 MUX required** = 3
- Number of inputs = 4
- Number of outputs = 1
- Select lines = 2

6. Interpretation of Result

- A **4:1 Multiplexer** is successfully implemented using **three 2:1 multiplexers**.
- The first stage selects between input pairs, and the second stage selects the final output.
- This hierarchical design reduces hardware complexity and is commonly used in digital systems.

Final Answer: A **4:1 Multiplexer** can be implemented using **three 2:1 Multiplexers**, with S_0 controlling the first stage and S_1 controlling the final selection.

Question 5: Design a Synchronous 3-bit Binary Counter Using JK Flip-Flops

1. Problem Understanding

Design a **3-bit synchronous binary counter** using **JK flip-flops**. In a synchronous counter, all flip-flops receive the **same clock pulse** and change state simultaneously.

2. Method Selection

For JK flip-flops:

- $J = K = 1 \rightarrow$ Toggle
- All flip-flops are connected to a common clock.

Input equations:

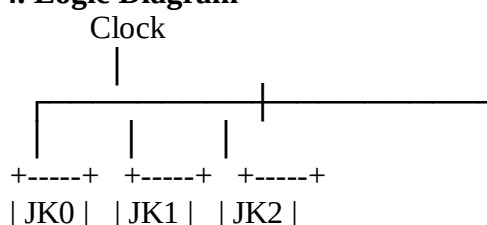
- $J_0 = K_0 = 1$
- $J_1 = K_1 = Q_0$
- $J_2 = K_2 = Q_0 \cdot Q_1$

3. State Table

Present State ($Q_2Q_1Q_0$) **Next State ($Q_2Q_1Q_0$)**

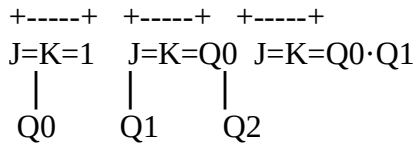
000	001
001	010
010	011
011	100
100	101
101	110
110	111
111	000

4. Logic Diagram



Parameter

Result



5. Counter Sequence

$000 \rightarrow 001 \rightarrow 010 \rightarrow 011 \rightarrow 100 \rightarrow 101 \rightarrow 110 \rightarrow 111 \rightarrow 000$

6. Accuracy of Design

- Flip-flops used: **3 JK Flip-Flops**
- Clock: **Common clock (Synchronous)**
- States: **8 (0 to 7)**
- Modulus (MOD): **8**

7. Interpretation of Result

- The counter increments by **1** on every clock pulse.
- Since all flip-flops are clocked simultaneously, the propagation delay is lower than in an asynchronous counter.
- The counter cycles through all **8 binary states** before repeating.

Final Answer

A **3-bit synchronous binary counter** is implemented using **three JK flip-flops** with:

- $J_0 = K_0 = 1$
- $J_1 = K_1 = Q_0$
- $J_2 = K_2 = Q_0 Q_1$

It counts from **000 to 111** (0 to 7) and then repeats, satisfying the required design.